

Algoritmos concurrentes

Quiz 4

En todos los casos, marque todas las respuestas que considere correctas.

1. ¿Cuál es el propósito de `suspend` en Kotlin?
 - a) Ejecuta una función automáticamente en otro *thread*.
 - b) Permite que una función entre en pausa sin bloquear un *thread*.
 - c) Acelera la ejecución de una función.
 - d) Previene excepciones.
2. ¿Qué biblioteca se usa comúnmente para las corrutinas en Kotlin?
 - a) `kotlin.async`.
 - b) `java.concurrent`.
 - c) `kotlinx.coroutines`.
 - d) `kotlin.parallel`.
3. ¿Qué hace `delay(1000)` dentro de una corrutina?
 - a) Bloquea el *thread* actual durante un segundo.
 - b) Pausa la corrutina sin bloquear el *thread*.
 - c) Detiene todas las corrutinas durante un segundo.
 - d) Crea un nuevo *thread*.
4. ¿Cuál es la principal diferencia entre `delay()` y `Thread.sleep()`?
 - a) `delay()` es más lento.
 - b) `Thread.sleep()` sólo funciona en corrutinas.
 - c) `delay()` no bloquea el hilo.
 - d) No hay diferencia.
5. ¿Cuál es el propósito de `runBlocking`?
 - a) Inicia un número infinito de corrutinas.
 - b) Impide la ejecución asíncrona.
 - c) Crea *threads* en segundo plano automáticamente.
 - d) Bloquea el *thread* actual hasta que las corrutinas finalicen.

6. Considere el siguiente código:

```
runBlocking {  
    launch {  
        delay(1000)  
        println("Campeón")  
    }  
    println("Belgrano")  
}
```

¿Qué se imprime primero?

- a) Belgrano.
- b) Las dos al mismo tiempo.
- c) Campeón.
- d) Nada.

7. ¿Cuál de las siguientes afirmaciones sobre las corrutinas es verdadera?

- a) Cada corrutina crea un nuevo *thread*.
- b) Las corrutinas solo pueden ejecutarse secuencialmente.
- c) Las corrutinas son más ligeras que los *threads*.
- d) Las corrutinas reemplazan todo el *multithreading*.

8. ¿Cuál es la principal ventaja de la programación no bloqueante?

- a) Reduce el uso de memoria a cero.
- b) Permite que los hilos continúen realizando otras tareas mientras esperan.
- c) Elimina la necesidad de funciones.
- d) Garantiza una CPU más rápida.

9. En la implementación bloqueante, ¿qué debería ocurrir cuando un consumidor hace `dequeue` sobre una cola vacía?

- a) El *thread* debería terminar.
- b) Se debería arrojar un error.
- c) El consumidor debería quedar esperando con un `Condvar`.
- d) Algún productor debería acudir al rescate colocando algo en la cola.

10. ¿Qué hace `compare_exchange`?

- a) Bloquea todos los *threads* temporalmente.
- b) Intercambia dos variables locales.
- c) Actualiza un valor sólo si coincide con el esperado.

- d) Libera memoria automáticamente según las reglas de *ownership*.

11. ¿Cuál de los siguientes problemas es típico en algoritmos lock-free?

- a) El problema *ABA*.
- b) *Deadlock* por múltiples *mutexes*.
- c) *Stack overflow*.
- d) Fragmentación de disco.

12. ¿Qué describe mejor el problema *ABA*?

- a) Un *thread* adquiere dos *locks* simultáneamente.
- b) Un valor cambia $A \rightarrow B \rightarrow A$ y esto provoca un uso innecesario de la CPU.
- c) Dos consumidores leen el mismo nodo simultáneamente.
- d) Un valor cambia $A \rightarrow B \rightarrow A$ y *CAS* no detecta el cambio.